Lab 1: Whitted Ray Tracing

Advanced Computer Graphics, 2026–2027
Stuart Lance Richards Weir (NIA 288001) · Gerard Borràs (NIA 266738)

1. Overview

We completed all five tasks: the image-painting loop, the Intersection, Depth and Normal integrators, the Phong BRDF, and a `WhittedIntegrator` that handles Phong, Mirror and Transmissive materials with shadows and an ambient term. Each integrator is its own class derived from `Shader`, and `main.cpp` only chooses which one to instantiate. All images below are our own renders at 720×512. Besides the required images we rendered parameter sweeps (depth range, ambient, shininess, number of lights, refractive ratio and recursion depth) to check that each term behaves as the theory predicts.

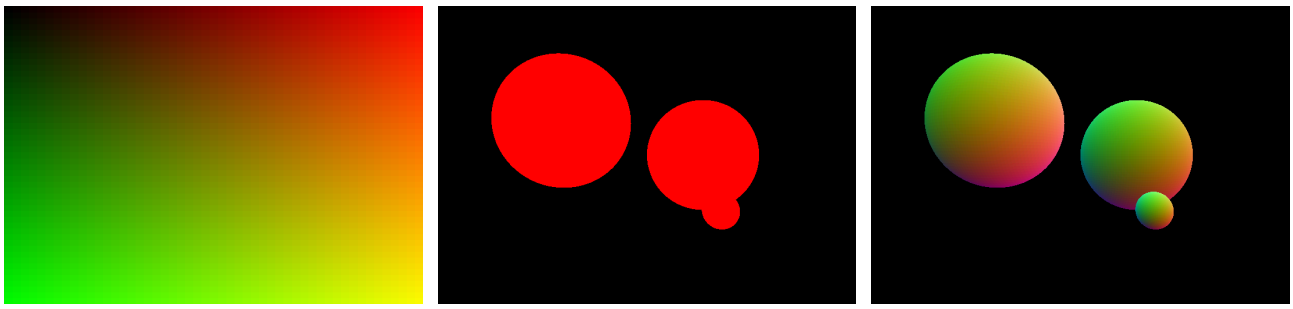
2. Painting the image and the simple integrators (Tasks 1–4)

Task 1. Each pixel stores $\left(\frac{x}{\text{width}}, \frac{y}{\text{height}}, 0\right)$. Red grows from left to right and green from top to bottom, so the corners are black, red, green and yellow (Figure 1). This confirmed the film's orientation: row 0 is the top of the image.

Task 2: Intersection. For every camera ray we only need to know *whether* it hits something, not *what* it hits first, so the shader uses the any-hit test `hasIntersection()`. That test can stop at the first object it finds, which makes it cheaper than searching for the closest hit. Hits are red and misses take the background colour.

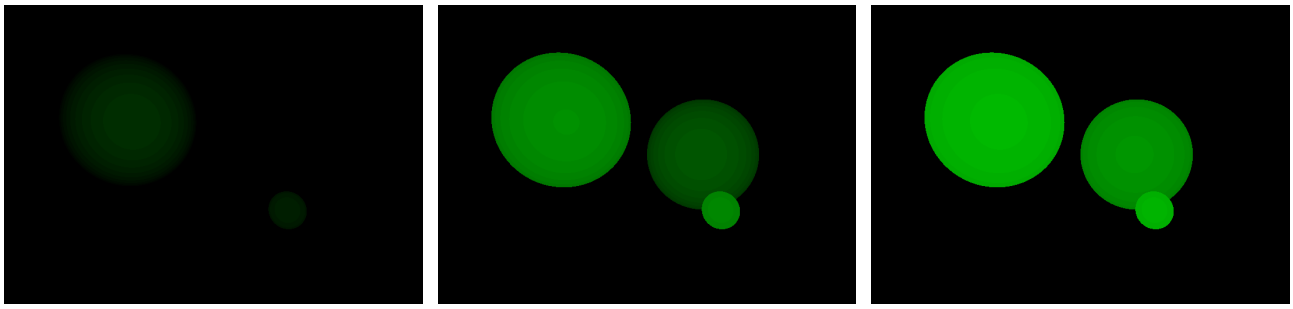
Task 3: Depth. Here the closest hit matters, so we use `getClosestIntersection()`, measure the distance d from the ray origin to the hit point, and scale the shader's colour (green) by $\max(1 - d/d_{\max}, 0)$. Brightness therefore falls linearly from the camera to $d_{\max}$, and anything beyond it is black. Figure 2 shows the role of $d_{\max}$: with 4 only the front of the nearest spheres is barely visible; with 7.5 the near/far ordering is clear; with 12 everything is bright and the depth contrast is compressed. $d_{\max}$ is effectively the depth range mapped onto the available brightness levels.

Task 4: Normal. The world-space unit normal at the closest hit has components in $[-1, 1]$. We map it to a colour with $(\boldsymbol{n} + 1)/2$, so each axis becomes one channel: surfaces facing up are green, facing right are red, and so on. The smooth gradient across each sphere (Figure 1) shows the normals vary continuously, which the shading models below depend on.



T1 gradient | T2 intersection | T4 normals

Figure 1: Tasks 1, 2 and 4: gradient, intersection and normal integrators.



$d_{\max} = 4$ | $d_{\max} = 7.5$ (default) | $d_{\max} = 12$

Figure 2: Task 3: depth integrator for three values of $d_{\max}$.

3. Phong material (Task 5.1)

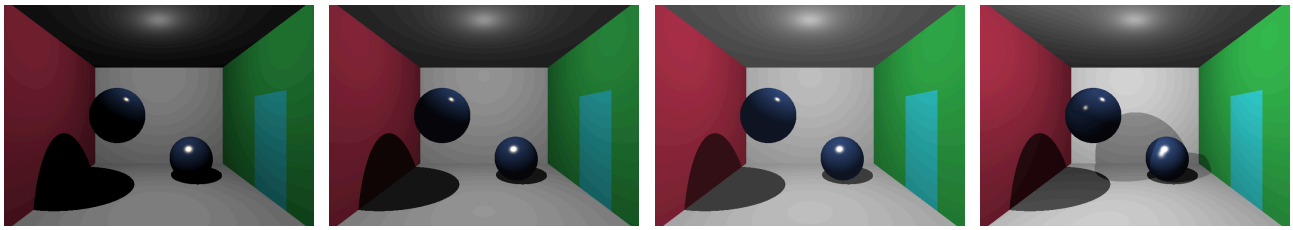
The Phong BRDF is $f_r = \rho_d/\pi + k_s \max(\boldsymbol{\omega}_o \cdot \boldsymbol{\omega}_r, 0)^{\alpha}$, with $\boldsymbol{\omega}_r = 2(\boldsymbol{n} \cdot \boldsymbol{\omega}_i)\boldsymbol{n} - \boldsymbol{\omega}_i$ the mirror direction of the incoming light. The diffuse part is constant: light is scattered equally in all directions, and the division by π keeps the BRDF from reflecting more energy than it receives (the cosine integrated over the hemisphere is π). The specular part is a lobe

around ω_r: it is brightest when we look straight along the reflected light, and α sets how quickly it falls off. We clamp $\omega_o \cdot \omega_r$ at 0 before raising it to α. Without the clamp, an even α turns a negative cosine into a false highlight on the wrong side of the lobe, and a fractional α produces NaN pixels.

The Cornell box shows both glossy materials side by side (Figure 3): the bottom-right sphere has $\alpha = 20$ and a wide, soft highlight, while the top-left one has $\alpha = 80$ and a small, sharp one, as from a more polished surface.

4. Direct illumination with point lights (Task 5.2)

For a Phong hit point x the integrator evaluates $L_o = a_t \rho_d + \sum_s L_i^s f_r(\boldsymbol{n}, \boldsymbol{\omega}_i^s, \boldsymbol{\omega}_o) \max(\boldsymbol{\omega}_i^s \cdot \boldsymbol{n}, 0) V_s(x)$, where $\boldsymbol{\omega}_o$ is the reversed camera ray. For each light it builds a **shadow ray** from x towards the light, limited to $t \in [\varepsilon, d_{\text{light}}]$. The upper bound means only objects *between* the point and the light can block it, not ones behind the light. The lower bound ε stops the ray from hitting the surface it starts on, which would otherwise shadow random points ("shadow acne") because of floating-point error. Visibility only needs a yes/no answer, so we use the cheaper any-hit test again. The cosine is clamped so a light behind the surface adds nothing instead of subtracting.



$a_t = 0$ | $a_t = 0.1$ (default) | $a_t = 0.3$ | $a_t = 0.1$, two lights

Figure 3: Task 5.2: ambient term a_t (0 matches the brief's Fig. 7) and a second light.

Ambient term. Whitted ray tracing ignores light that bounces between surfaces, so with $a_t = 0$ every shadow is pure black (Figure 3), which no real room looks like. The ambient term $a_t \rho_d$ is a constant, cheap stand-in for that indirect light. At 0.1 the shadows keep the colour of the surface they fall on. At 0.3 the image looks washed out because the ambient light is the same everywhere, which flattens the contrast that gives the scene its shape.

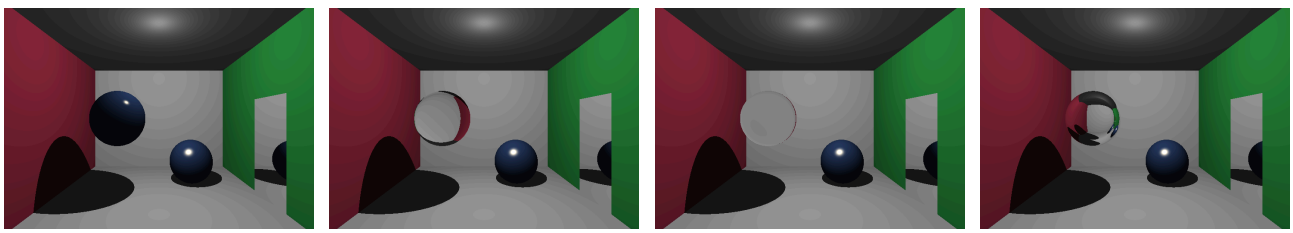
Several lights. Adding a second, weaker light at $(-2.5, 1.5, 1)$ gives each sphere two highlights and two shadows. Where the shadows overlap only the ambient term remains; where just one light is blocked the floor is partly lit. This confirms the contributions are summed over all lights, with the ambient term added once per point rather than once per light.

5. Mirror material (Task 5.3)

A perfect mirror has no BRDF lobe of its own: all the light leaving it in direction $\boldsymbol{\omega}_o$ arrives from the single mirror direction. So instead of evaluating a BRDF, the integrator computes $\boldsymbol{\omega}_r = \boldsymbol{d} - 2(\boldsymbol{n} \cdot \boldsymbol{d})\boldsymbol{n}$ from the incoming ray direction $\boldsymbol{d}$, traces a new ray from the hit point, and returns whatever colour that ray finds (Figure 4). This is what makes the method recursive. Each new ray carries `depth + 1`, and the integrator returns black once the depth passes a maximum (10). Without that limit, two mirrors facing each other would recurse until the program crashes.

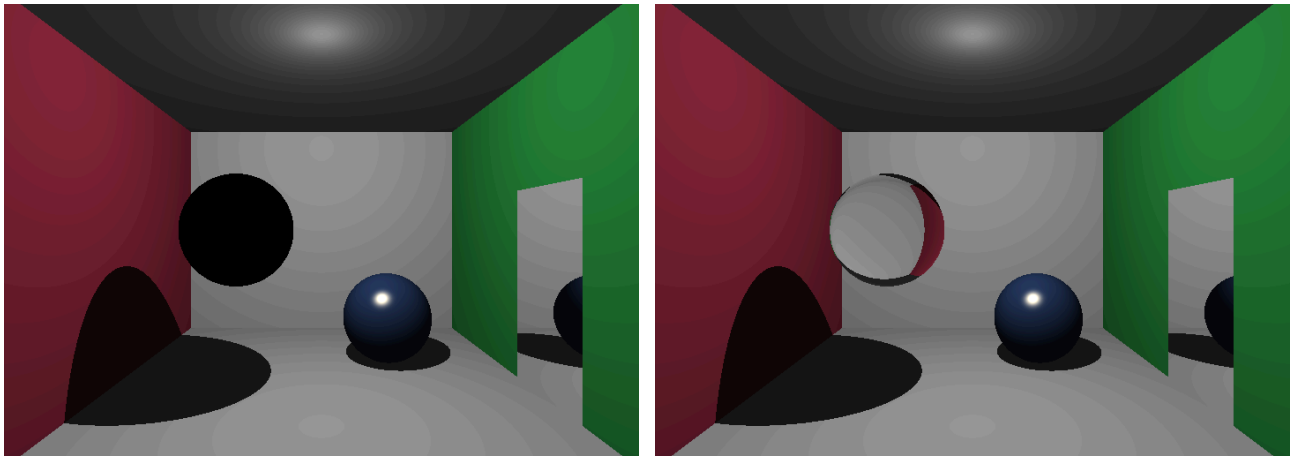
6. Transmissive material (Task 5.4)

A transmissive surface bends the ray instead of reflecting it. The material only stores $\mu_t = \eta_1/\eta_2$; the integrator uses Snell's law in vector form (Eq. 8): $\boldsymbol{\omega}_t = -\mu_t \boldsymbol{\omega}_o + \boldsymbol{n}\left(\mu_t(\boldsymbol{n} \cdot \boldsymbol{\omega}_o) - \sqrt{1 - \mu_t^2\left(1 - (\boldsymbol{n} \cdot \boldsymbol{\omega}_o)^2\right)}\right)$. Sphere normals always point outwards, so the sign of $\boldsymbol{n} \cdot \boldsymbol{\omega}_o$ tells us which side we are on. If it is negative the ray is inside, travelling from glass to air, so we flip the normal and use $1/\mu_t$. If the square root's argument is negative there is no refracted ray (**total internal reflection**), and we reflect the ray as a mirror would.



mirror only | $\mu_t = 0.7$ (final) | $\mu_t = 0.9$ | $\mu_t = 1.3$

Figure 4: Tasks 5.3–5.4: mirror on the right wall and the glass sphere for several μ_t.



max depth 1 max depth 2

Figure 5: Glass sphere ($\mu_t = 0.7$) with maximum depth 1 and 2.

Refractive ratio. With $\mu_t = 0.7$ (roughly air to glass) the sphere acts as a lens: the scene behind it appears upside down and mirrored, so the red wall shows up on its *right* side (Figure 4). At $\mu_t = 0.9$ the rays bend less, the image is barely distorted and the sphere almost disappears into the wall behind it. With $\mu_t = 1.3$ the sphere is *less* dense than its surroundings (like an air bubble in water); rays reaching it at grazing angles undergo total internal reflection, so a ring around the edge reflects the coloured walls.

Recursion depth. Seeing through the sphere takes two refractions, one entering and one leaving. With a maximum depth of 1 the exit ray is cut off and the sphere renders black; from 2 upwards it looks correct (Figure 5).

7. Debugging note

Rendering on macOS crashed at random when the image was saved. AddressSanitizer showed the BMP writer copying 8-byte `long` header fields into a 14-byte buffer: `long` is 4 bytes on Windows but 8 on macOS and Linux. We changed the header fields to fixed 32-bit integers, which also produces correct BMP headers on every platform. We used Claude (an AI assistant) to help diagnose this crash and implement the fix.